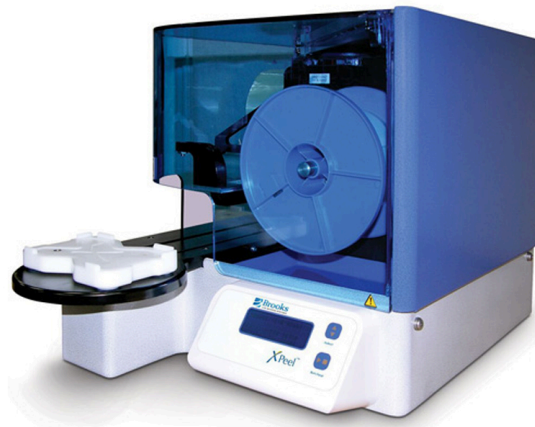


Coding Challenge

Device Integrations Engineer — Take-Home Challenge

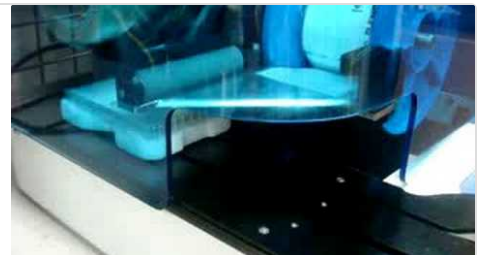
XPeel Driver Implementation



XPeel

The XPeel automatically removes seals from microplates.

 <https://www.youtube.com/watch?v=XNb1Lbg6SgI>



Overview

XPeel is a device commonly used in Automata's system to peel a foil/seal from a microplate. It communicates over an RS-232 (serial) interface using a simple text-based command/response protocol.

This exercise asks you to build a small Python driver for XPeel. It's designed to reflect the kind of work you'd do in this role day to day: reading a real instrument manual, implementing a serial protocol correctly, and writing code that's structured well enough for a teammate to extend.

Expected time: 2–3 hours. We are not expecting production polish. We're looking at how you approach the problem, not how much time you sink into it. If you find yourself at 4+ hours, stop and write up what you'd do next in the README instead.

You may use an LLM if you choose so. We have access to AI tools as part of the role, so we think it is only fair you undertake this exercise with the same access and tools we have at Automata.

You may access and use the full XPeel user manual to complete the challenge. We provide a few specific pages for your convenience.

Reference material

- Full XPeel user manual: link to manual: <https://web.azenta.com/hubfs/azenta-files/resources/manuals-guides/440140-Xpeel-Manual.pdf>
 - Command reference starts on page 46; unsolicited message behavior is documented on page 53.
- pySerial docs: <https://pyserial.readthedocs.io/>

Protocol snippet

From page 46 of the manual — command which triggers the peel process:

Automated Plate Seal Remover Request:

Command:

```
*xpeel:AB<CR><LF>
```

The variable 'A' refers to the set of parameters: 1-9

The 'xpeel' operation has a number of parameters that can be selected to achieve better performance on difficult or unusual seal types. For most seals the default and fastest settings will give good performance. In an automated environment it may be advisable to be more conservative, or to provide for a retry at slower settings when the Automated Plate Seal Remover detects that a seal is not removed. Refer to the manual for techniques in determining proper set up parameters.

Parameter sets 1-8 are fixed; parameter set 9 can be set and saved manually by the user.

Set Number	Begin Peel Location	Speed
Set 1	Default -2 mm	fast
Set 2	Default -2 mm	slow
Set 3	Default	fast
Set 4	Default	slow
Set 5	Default +2 mm	fast
Set 6	Default +2 mm	slow
Set 7	Default +4 mm	fast
Set 8	Default +4 mm	slow
Set 9	custom	custom

The second term, variable 'B,' refers to adhere time in seconds

- 1=2.5 seconds
- 2=5 seconds
- 3=7.5 seconds
- 4=10 seconds

A typical command would be:

```
*xpeel:41<CR><LF>
```

This command is a request for the Automated Plate Seal Remover to move the conveyor to the default starting position for adhesion, move the elevator down, wait 2.5 seconds adhere time, Automated Plate Seal Remover with a slow speed, and finally return the plate to the user. The 'xpeel' command is to be issued only after a sealed plate is placed on the conveyor.

Acknowledgment:

```
*ack:<CR><LF>
```

Normal Response:

```
*ready:00,00,00<CR><LF>
```

pySerial usage

Example usage of pySerial library:

Open port at "9600,8,N,1", no timeout:

```
>>> import serial
>>> ser = serial.Serial('/dev/ttyUSB0') # open serial port
>>> print(ser.name)                    # check which port was really use
d
>>> ser.write(b'hello')                # write a string
>>> ser.close()                        # close port
```

The Challenge

Part 1 — Basic peel command

Implement a Python class representing the XPeel device with a `peel()` method that triggers the peeling process and returns once the plate has been peeled (i.e. once the normal `*ready:` response is received).

Use pySerial. Assume the port is opened at 9600,8,N,1.

Part 2 — Configurable parameters

Extend `peel()` so the command's parameters (A and B) can be specified by the caller, with defaults matching the documented example command (`*xpeel:41`).

Part 3 — A second command

Implement a second method for the other peel-related command documented in the manual (tape-left).

Part 4 — Unsolicited messages

Per page 53 of the manual, XPeel can send messages on its own, independent of the command/response exchange. Update your driver's initialization so that any unsolicited messages present on the line are correctly consumed before normal command/response operation begins.

Part 5 — Error handling

Your driver should explicitly handle, at minimum:

- No response received within a timeout window — choose and justify a timeout value.
- A malformed or unexpected response.
- Receiving `ack:` but never a subsequent `ready:`.

For each case, decide what your driver should do (raise a specific exception, retry, log, etc.).

Part 6 — Tests

Include at least one test that verifies correct behavior on a normal peel cycle, and one that verifies your timeout/error handling. You do not need real hardware.

Part 7 — README

Include a short README (bullet points are fine) covering:

- Any assumptions you made.
- Tradeoffs in your design and why you made them.

- What you'd do differently, or add, with more time.

Submission

Please submit a link to a GitHub repository (public or with access granted) containing your code, tests, and README. Commit history doesn't need to be tidy so feel free to commit as you go.

Questions about the protocol or ambiguities in the manual? Please reach out. How you handle ambiguity, and whether you ask about it, is itself part of what we're evaluating.